

TRANSCRIÇÃO – AULA 5- ACTIONS

Olá! Dando continuidade ao nosso curso sobre GitHub Actions, vamos explorar os “jobs” ou trabalhos.

Você vai entender como eles funcionam e podem ser configurados para automatizar nossas tarefas.

No GitHub Actions, um “workflow” ou um fluxo de trabalho, ele é composto por um ou mais “jobs”, trabalhos.

Cada job contém uma série de etapas ou “steps” que são executadas de forma sequencial.

Essas etapas compartilham o mesmo ambiente de execução, incluindo o sistema de arquivos, o que significa que arquivos criados ou até mesmo modificados em uma etapa estão disponíveis para as próximas etapas dentro do mesmo job.

Vejamos um outro exemplo com a ajuda do GitHub Copilot.

Eu estou aqui com o meu Visual Studio Code aberto.

Inclusive eu criei um arquivo aqui chamado “exemplo-13.yaml”.

E nele tem alguns jobs aqui.

Um deles aqui está na linha seis, que é o “test” e outra que na linha 24 que é o “deploy”.

Então note que aqui nós temos inúmeros “steps” ou tarefas onde eu estou, por exemplo, na parte de testes eu estou justamente chamando a última versão do node js, que é o 20.

Eu estou pedindo para que essa tarefa execute as dependências do node, tanto que está aqui na linha 19, já aqui, na linha 24 nós temos uma outra tarefa que é o “deploy”.

Nesse caso eu estou pedindo para que ele pega essa aplicação e implemente na web, no caso executando o comando “npm run deploy”.

Mas ficou um pouco complicado de entender o que esse arquivo está fazendo?

Não tem problema, podemos fazer novamente o uso do GitHub Copilot.

Então vamos selecionar aqui o código.

Botão direito, Copilot, “Start in editor”, e vou perguntar o seguinte:

“Poderia me explicar o que está acontecendo aqui?”. Dá o enter.

Ele já leu o arquivo.

Olha que legal!

Para cada etapa que está acontecendo nesse job, ele está me explicando o que está acontecendo aqui, por exemplo, ele já identificou que na primeira linha a gente está definindo o nome desse fluxo de trabalho que vai se chamar “CI/CD Workflow”, inclusive você pode escolher outros nomes tal que você desejar.

Sequencialmente, já está explicando aqui que a segunda linha especifica quando que o fluxo de trabalho será acionado.

Nesse caso, quando ocorrer o push no repositório, aqui já identificou que temos a seção “jobs” que define os trabalhos a ser executados no fluxo de trabalho.

Ele já identificou que tem dois tipos de jobs aqui nesse caso, o “test” será executado na máquina virtual Ubuntu, que é o mais recente, que é “ubuntu-latest”.

Dentro do teste, a gente está executando algumas tarefas ou “steps”, né? Etapas que serão executadas sequencialmente.

Nesse caso, estamos instalando aqui as dependências do node, aí vamos avançar um pouquinho mais para ver o que o Copilot nos ensinou aqui.

Olha que legal! Aqui no ponto 11, ele identificou para mim que nós estamos executando um outro job aqui, que é o “deploy” que será executado também na máquina virtual do Ubuntu e que está sendo definida com o nome de “test”.

Então isso significa que o trabalho “deploy” só será executado se o primeiro step, que é o test, for concluído com sucesso.

Você vê que são coisas bem sequenciais, assim como o trabalho test, o trabalho deploy também tem uma série de etapas que serão executadas sequencialmente.

E por último, aqui a gente vai chegando até o finalzinho aqui no item 14, a segunda etapa, chamada de “Deploy para produção”, executa o comando “npm run deploy” para realizar o deploy do projeto para justamente disponibilizar em produção.

Então veja aqui, que de fato, o Copilot, ele conseguiu explicar detalhadamente cada linha o que está acontecendo aqui e serve até como parâmetro de estudos para você aprender o que está acontecendo aqui e até documentação.

Nós temos aqui ajuda justamente dessa ferramenta incrível para nos auxiliar no nosso dia a dia.

Se você quiser saber mais sobre dependências de jobs, consulte a seção “Como criar jobs dependentes” na documentação do GitHub Actions.

No próximo vídeo vamos ver como configurar jobs dependentes na prática.

Até já!